

Retrieval-Augmented Generation (RAG) for Answering Multiple-Choice Questions (MCQs)

A Dense Wikipedia Retriever Coupled with Phi-3-mini on the Smart MCQ Solver Challenge (Kaggle)

Naini Diwan

July 2026

Abstract

We present a retrieval-augmented generation (RAG) approach to multiple-choice question answering, evaluated on the Smart MCQ Solver Challenge hosted by IIT Madras on Kaggle. The system pairs a dense passage retriever, a *MiniLM* sentence encoder indexed with *FAISS* over a *Wikipedia STEM* corpus. The frozen instruction-tuned language model, *Phi-3-mini-4k-instruct* was prompted to rank three most plausible answers per question. On a 200-question evaluation sample drawn from the labeled training set, the pipeline achieved **61.0% top-1 accuracy, macro F1-score of 0.5996, and MAP@3 of 0.7625**. Because of existing constraints of GPU on Kaggle, retrieval was run over a 5,000-passage subsample (not the full corpus), and evaluation was conducted on a 10% of the training data.

1. Introduction

Answering MCQs over knowledge-intensive questions is difficult for a model to solve from parametric knowledge alone, particularly for a small instruction-tuned language model without task-specific fine-tuning. RAG addresses this by supplying the model with relevant supporting text at inference time, shifting some of the burden from memorized parameters to an external, searchable corpus. The submissions are scored by Mean Average Precision at 3 (MAP@3), rewarding a ranked list of the three most likely correct answers rather than a single prediction.

Unlike a fine-tuning approach, which adapts model weights to the task's training distribution, this pipeline uses no trainable parameters: the sentence encoder and language model are used off-the-shelf, and all task adaptation happens through retrieval and prompt construction.

2. System Architecture

Component	Implementation
Corpus	STEM Wikipedia dump (mbanaei/stem-wiki-cohere-no-emb), 5,000-passage random subsample
Retriever	all-MiniLM-L6-v2 sentence embeddings (384-dim) + FAISS IndexFlatIP (cosine via normalized inner product)
Generator	microsoft/Phi-3-mini-4k-instruct, deterministic decoding
Answer parsing	Character-level scan of generated text for valid options, order-preserving, deduplicated, padded to length 3

3. Data

3.1 Competition data

Question data is loaded from the competition's train.csv and test.csv, each row containing a prompt, five answer options (A–E), and, for the training set, the correct answer label.

3.2 Knowledge corpus

The auxiliary Kaggle dataset *stem-wiki-cohere-no-emb* is used in the retrieval pipeline. The STEM Wikipedia dump used contains the columns `id`, `title`, `text`, `url`, `wiki_id`, `views`, `paragraph_id`, and `langs`, loaded via the Hugging Face datasets library from Arrow shard files and converted to a pandas DataFrame. Rows with a missing text field are dropped.

For the retrieval index actually built, corpus is reduced to a random sample of 5,000 passages rather than the full dump. A full-scale indexing of a large Wikipedia dump would have exceeded Kaggle's session limits.

4. Retrieval Method

Passage text from the 5,000-row sample is embedded using the all-MiniLM-L6-v2 sentence-transformer model, in batches of 256, with embeddings L2-normalized at encoding time. Because embeddings are normalized, inner-product search is mathematically equivalent to cosine similarity, allowing the notebook to use FAISS's IndexFlatIP (a flat, exact inner-product index) rather than a cosine-specific index type. The resulting index (5,000 vectors, dimension 384) and its associated passage metadata are persisted to disk as `wiki.index` and `wiki_meta.parquet`, so the embedding step does not need to be repeated if the notebook kernel restarts. At query time, a `retrieve(query, k)` function embeds the incoming question text with the same encoder and normalization, searches the FAISS index, and returns the raw text of the 3 highest-scoring passages. The retrieval is a single dense nearest-neighbor lookup per question.

5. Generation and Prompting Method

5.1 Model

Phi-3-mini-4k-instruct is loaded with `AutoModelForCausalLM`, using `bfloat16` precision on GPU (`float32` as a CPU fallback) and `device_map` set to place the model on the available device.

5.2 Prompt construction

For each question, a single prompt is built containing: the retrieved passages formatted as a bulleted context block, the question text, all five answer options labeled A through E, and an explicit instruction to output only the three most likely option letters, ranked best-first and comma-separated.

```
def build_prompt(row, passages):
    context = "\n\n".join(f"- {p}" for p in passages)
    options_text = "\n".join(f"{c}. {row[c]}" for c in option_cols)
    return f"""Use the context below to answer the multiple choice question.
Context: {context}
Question: {row['prompt']}
Options: {options_text}
Respond with ONLY the ranked letters of the three most likely correct options,
best first, separated by commas. Example: "B, D, A"
Answer: """
```

5.3 Decoding and answer extraction

Generation uses greedy decoding (`do_sample=False`) with a small output budget (`max_new_tokens=10`), appropriate given the expected output is a handful of letters rather than free-form text. The model's chat template is applied via `tokenizer.apply_chat_template` so the prompt is formatted according to Phi-3's

expected instruction format before generation. The generated text is parsed by scanning its upper-cased characters left to right, collecting each valid option letter (A–E) the first time it appears and skipping repeats. If the model returns fewer than three distinct valid letters, for example due to truncated or malformed output, the remaining option letters are appended in a fixed order (A, B, C, D, E) until three letters are present. This guarantees every prediction has exactly three ranked letters regardless of generation quality, at the cost of potentially padding with arbitrary, non-predictive letters when the model's own output is ambiguous.

6. Results

Three metrics are computed:

- Accuracy: exact match between the model's first-ranked (top-1) answer and the true label.
- Macro F1-score: F1 averaged evenly across the five answer classes, unweighted by class frequency.
- MAP@3: for each question, a score of $1/\text{rank}$ is awarded when the true answer appears in the top-3 predicted ranking (1.0 if ranked first, 0.5 if second, 0.333 if third, 0 otherwise), averaged across all evaluated questions. This mirrors the competition's own scoring metric.

On the 200-question evaluation sample, the pipeline achieved the following:

Metric	Value
Accuracy (top-1)	0.6100
Macro F1-score	0.5996
MAP@3	0.7625

The gap between top-1 accuracy (0.610) and MAP@3 (0.7625) indicates that in a meaningful share of cases where model's first-ranked answer was wrong, the correct answer still appeared in its second or third-ranked choice, consistent with the ranked-output design of both the prompt and the MAP@3 metric.

7. Limitations

- Corpus coverage: retrieval index is built over a 5,000-passage sample of the Wikipedia dump, which likely constrains recall for questions whose supporting text falls outside the sample.
- Evaluation sample size: metrics are computed on 200 of 2,000 available labeled training rows (10%), which narrows the confidence one can place in the reported metrics.
- No retrieval-only diagnostics: the notebook does not report retrieval-specific metrics (e.g., whether the correct answer's supporting passage was actually retrieved), making it difficult to separate retrieval quality from the language model's reasoning over the given context.
- Fixed k and no re-ranking: retrieval always returns exactly 3 passages by default with no query reformulation, hybrid search, or cross-encoder re-ranking step, any of which are common ways to improve RAG retrieval precision.
- Answer-parsing fallback: when generation fails to produce three distinct valid letters, the parser pads the ranking with arbitrary remaining letters in fixed A–E order, which is not a modeled prediction and could understate or overstate performance depending on how often the fallback triggers (frequency not reported).